

ALGORITHM

1. Main Program Flow

1. Start the program.
2. Display the menu options (1 to 5) repeatedly using an infinite loop.
3. Read user's choice.
4. Execute corresponding function using switch-case statement.
5. If choice is 5, exit the program loop.

2. Check Digits in Increasing Order

1. Input a positive integer num.
2. Initialize `prevDigit = 10` and `isIncreasing = 1`.
3. Extract last digit `currDigit = num % 10`.
4. If `currDigit >= prevDigit`, set `isIncreasing = 0` and break loop.
5. Update `prevDigit = currDigit` and `num = num / 10`. Repeat until `num == 0`.
6. Print result based on `isIncreasing` flag.

3. Check Digits in Decreasing Order

1. Input a positive integer num.
2. Initialize `prevDigit = -1` and `isDecreasing = 1`.
3. Extract last digit `currDigit = num % 10`.
4. If `currDigit <= prevDigit`, set `isDecreasing = 0` and break loop.
5. Update `prevDigit = currDigit` and `num = num / 10`. Repeat until `num == 0`.
6. Print result based on `isDecreasing` flag.

4. Find Largest and Smallest Digit

1. Input a positive integer num.
2. Initialize `maxDigit = 0` and `minDigit = 9`.
3. Extract digit `d = num % 10`.
4. If `d > maxDigit`, set `maxDigit = d`.
5. If `d < minDigit`, set `minDigit = d`.
6. Reduce `num = num / 10` and repeat until `num == 0`.
7. Print `maxDigit` and `minDigit`.

5. Find Second Largest and Second Smallest Digit

1. Input a positive integer num.
2. Create a frequency array `present[10] = {0}`.

3. Mark digits present in num as `present[digit] = 1`.
4. Traverse array from index 9 down to 0 to find the 2nd present digit for Second Largest.
5. Traverse array from index 0 up to 9 to find the 2nd present digit for Second Smallest.
6. Print the second largest and second smallest digits.

TEST CASES

Test Case #	Menu Option Selected	Input Number	Expected Result
TC-01	1 (Check Increasing)	13579	Digits are in increasing order
TC-02	1 (Check Increasing)	13357	Digits are NOT in strictly increasing order
TC-03	2 (Check Decreasing)	97521	Digits are in decreasing order
TC-04	2 (Check Decreasing)	98842	Digits are NOT in strictly decreasing order
TC-05	3 (Max & Min Digit)	48205	Largest: 8, Smallest: 0
TC-06	4 (2nd Max & 2nd Min)	948125	2nd Largest: 8, 2nd Smallest: 2
TC-07	4 (2nd Max & 2nd Min)	555	Insufficient unique digits

C PROGRAM

```
#include <stdio.h>
#include <stdlib.h>

void checkIncreasing();
void checkDecreasing();
void findMaxMin();
void findSecondMaxMin();

int main() {
    int choice;

    while (1) {
        printf("
=== MENU ===
");
        printf("1. Check Digits in Increasing Order
");
        printf("2. Check Digits in Decreasing Order
");
        printf("3. Find Largest and Smallest Digit
");
        printf("4. Find Second Largest and Second Smallest Digit
");
        printf("5. Exit
");
        printf("Enter choice: ");
        if (scanf("%d", &choice) != 1) {
            break;
        }

        switch (choice) {
            case 1:
                checkIncreasing();
                break;
            case 2:
                checkDecreasing();
                break;
            case 3:
                findMaxMin();
                break;
            case 4:
                findSecondMaxMin();
                break;
            case 5:
                exit(0);
            default:
                printf("Invalid choice!
");
        }
    }
    return 0;
}
```

```

}

void checkIncreasing() {
    long long num, temp;
    printf("Enter positive integer: ");
    scanf("%lld", &num);

    if (num <= 0) {
        printf("Invalid input!
");
        return;
    }

    temp = num;
    int prevDigit = 10;
    int isIncreasing = 1;

    while (temp > 0) {
        int currDigit = temp % 10;
        if (currDigit >= prevDigit) {
            isIncreasing = 0;
            break;
        }
        prevDigit = currDigit;
        temp /= 10;
    }

    if (isIncreasing) {
        printf("Digits are in strictly INCREASING order.
");
    } else {
        printf("Digits are NOT in strictly increasing order.
");
    }
}

void checkDecreasing() {
    long long num, temp;
    printf("Enter positive integer: ");
    scanf("%lld", &num);

    if (num <= 0) {
        printf("Invalid input!
");
        return;
    }

    temp = num;
    int prevDigit = -1;
    int isDecreasing = 1;

    while (temp > 0) {
        int currDigit = temp % 10;
        if (currDigit <= prevDigit) {
            isDecreasing = 0;

```

```

        break;
    }
    prevDigit = currDigit;
    temp /= 10;
}

if (isDecreasing) {
    printf("Digits are in strictly DECREASING order.
");
} else {
    printf("Digits are NOT in strictly decreasing order.
");
}
}

void findMaxMin() {
    long long num, temp;
    printf("Enter positive integer: ");
    scanf("%lld", &num);

    if (num <= 0) {
        printf("Invalid input!
");
        return;
    }

    temp = num;
    int maxDigit = 0, minDigit = 9;

    while (temp > 0) {
        int d = temp % 10;
        if (d > maxDigit) maxDigit = d;
        if (d < minDigit) minDigit = d;
        temp /= 10;
    }

    printf("Largest Digit : %d
", maxDigit);
    printf("Smallest Digit : %d
", minDigit);
}

void findSecondMaxMin() {
    long long num, temp;
    printf("Enter positive integer: ");
    scanf("%lld", &num);

    if (num <= 0) {
        printf("Invalid input!
");
        return;
    }

    int present[10] = {0};
    temp = num;

```

```

while (temp > 0) {
    present[temp % 10] = 1;
    temp /= 10;
}

int count = 0;
int secondMax = -1, secondMin = -1;

for (int i = 9; i >= 0; i--) {
    if (present[i]) {
        count++;
        if (count == 2) {
            secondMax = i;
            break;
        }
    }
}

count = 0;
for (int i = 0; i <= 9; i++) {
    if (present[i]) {
        count++;
        if (count == 2) {
            secondMin = i;
            break;
        }
    }
}

if (secondMax != -1) {
    printf("Second Largest Digit : %d", secondMax);
} else {
    printf("Second Largest Digit : N/A");
}

if (secondMin != -1) {
    printf("Second Smallest Digit : %d", secondMin);
} else {
    printf("Second Smallest Digit : N/A");
}
}

```

OUTPUT

```

=== MENU ===
1. Check Digits in Increasing Order
2. Check Digits in Decreasing Order
3. Find Largest and Smallest Digit

```

```
4. Find Second Largest and Second Smallest Digit
5. Exit
Enter choice: 1

Enter positive integer: 13579
Digits are in strictly INCREASING order.

=== MENU ===
1. Check Digits in Increasing Order
2. Check Digits in Decreasing Order
3. Find Largest and Smallest Digit
4. Find Second Largest and Second Smallest Digit
5. Exit
Enter choice: 2

Enter positive integer: 97521
Digits are in strictly DECREASING order.

=== MENU ===
1. Check Digits in Increasing Order
2. Check Digits in Decreasing Order
3. Find Largest and Smallest Digit
4. Find Second Largest and Second Smallest Digit
5. Exit
Enter choice: 3

Enter positive integer: 48205
Largest Digit : 8
Smallest Digit : 0

=== MENU ===
1. Check Digits in Increasing Order
2. Check Digits in Decreasing Order
3. Find Largest and Smallest Digit
4. Find Second Largest and Second Smallest Digit
5. Exit
Enter choice: 4

Enter positive integer: 948125
Second Largest Digit : 8
Second Smallest Digit : 2

=== MENU ===
1. Check Digits in Increasing Order
2. Check Digits in Decreasing Order
3. Find Largest and Smallest Digit
4. Find Second Largest and Second Smallest Digit
5. Exit
Enter choice: 5
```